

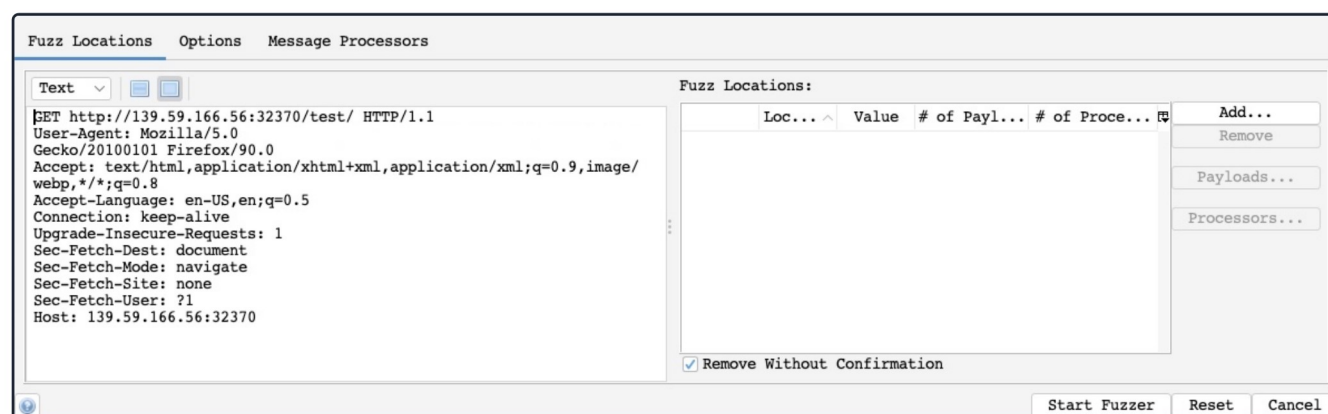
# ZAP Fuzzer

ZAP's Fuzzer is called (**ZAP Fuzzer**). It can be very powerful for fuzzing various web end-points, though it is missing some of the features provided by Burp Intruder. ZAP Fuzzer, however, does not throttle the fuzzing speed, which makes it much more useful than Burp's free Intruder.

In this section, we will try to replicate what we did in the previous section using ZAP Fuzzer to have an "apples to apples" comparison and decide which one we like best.

## Fuzz

To start our fuzzing, we will visit the URL from the exercise at the end of this section to capture a sample request. As we will be fuzzing for directories, let's visit `<http://SERVER_IP:PORT/test/>` to place our fuzzing location on **test** later on. Once we locate our request in the proxy history, we will right-click on it and select (**Attack>Fuzz**), which will open the **Fuzzer** window:



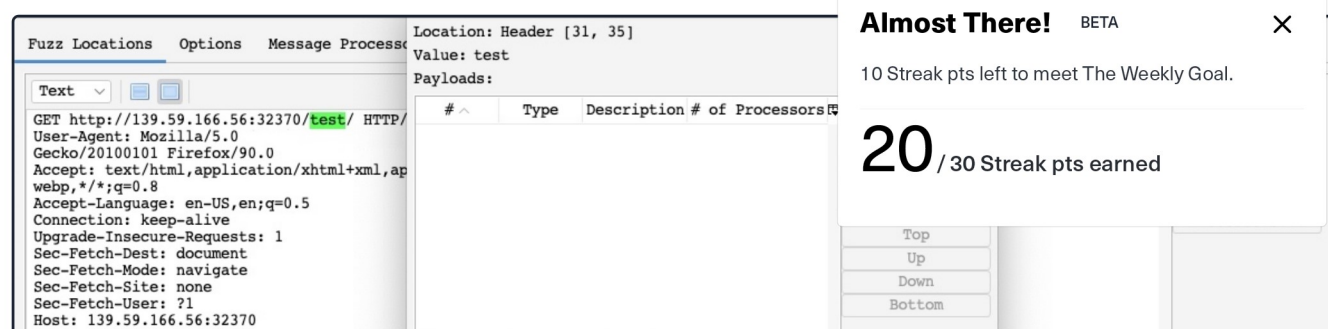
The main options we need to configure for our Fuzzer attack are:

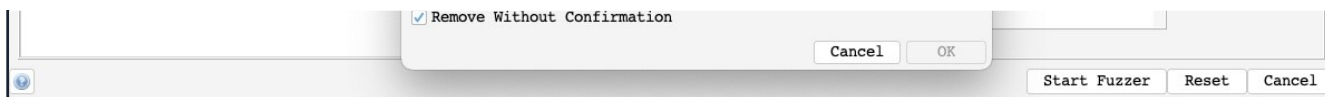
- Fuzz Location
- Payloads
- Processors
- Options

Let's try to configure them for our web directory fuzzing attack.

## Locations

The **Fuzz Location** is very similar to **Intruder Payload Position**, where our payloads will be placed. To place our location on a certain word, we can select it and click on the **Add** button on the right pane. So, let's select **test** and click on **Add**:





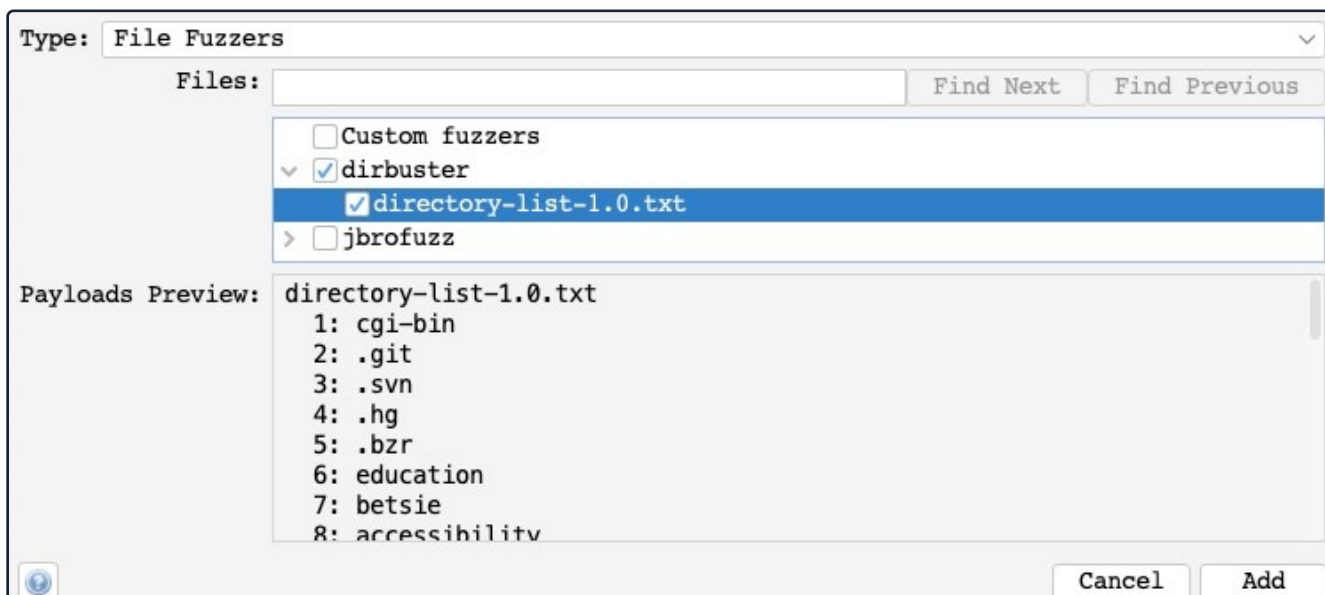
As we can see, this placed a **green** marker on our selected location and opened the **Payloads** window for us to configure our attack payloads.

## Payloads

The attack payloads in ZAP's Fuzzer are similar in concept to Intruder's Payloads, though they are not as advanced as Intruder's. We can click on the **Add** button to add our payloads and select from 8 different payload types. The following are some of them:

- **File**: This allows us to select a payload wordlist from a file.
- **File Fuzzers**: This allows us to select wordlists from built-in databases of wordlists.
- **Numberzz**: Generates sequences of numbers with custom increments.

One of the advantages of ZAP Fuzzer is having built-in wordlists we can choose from so that we do not have to provide our own wordlist. More databases can be installed from the ZAP Marketplace, as we will see in a later section. So, we can select **File Fuzzers** as the **Type**, and then we will select the first wordlist from **dirbuster**:



Once we click the **Add** button, our payload wordlist will get added, and we can examine it with the **Modify** button.

## Processors

We may also want to perform some processing on each word in our payload wordlist. The following are some of the payload processors we can use:

- Base64 Decode/Encode
- MD5 Hash
- Postfix String
- Prefix String
- SHA-1/256/512 Hash
- URL Decode/Encode
- Script

### Almost There!

BETA



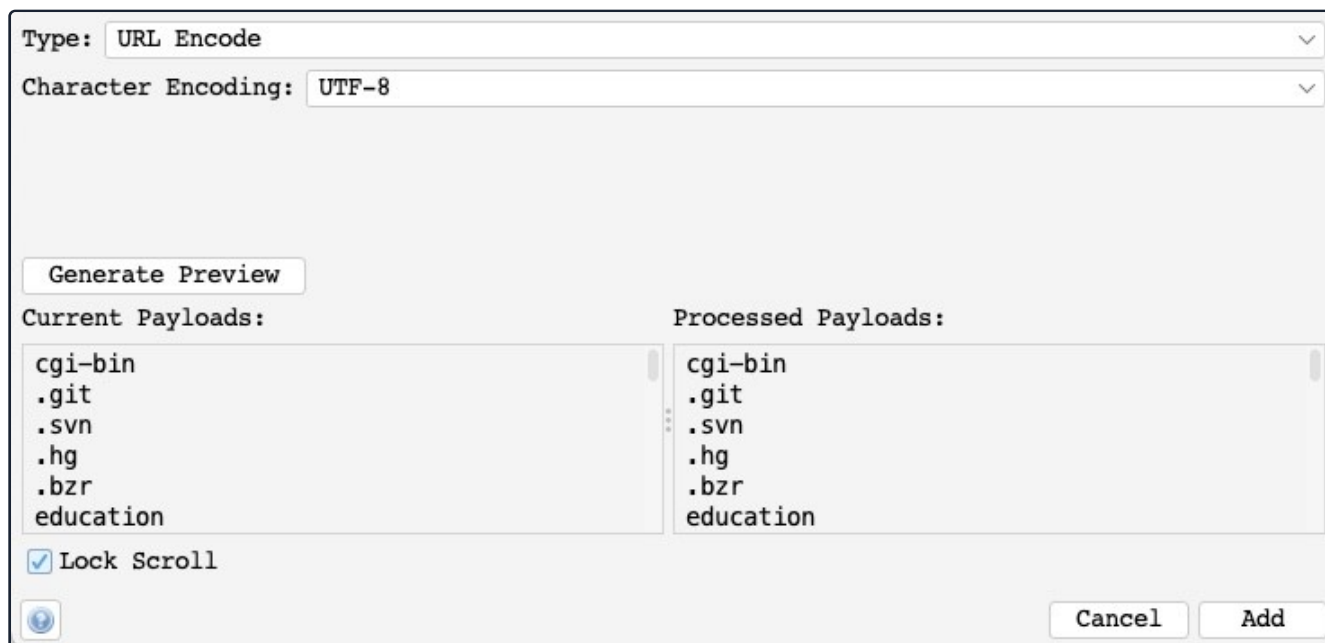
10 Streak pts left to meet The Weekly Goal.

**20** / 30 Streak pts earned

As we can see, we have a variety of encoders and hashing algorithms to select from. We can also add a custom string before the

payload with **Prefix String** or a custom string with **Postfix String**. Finally, the **Script** type allows us to select a custom script that we built and run on every payload before using it in the attack.

We will select the **URL Encode** processor for our exercise to ensure that our payload gets properly encoded and avoid server errors if our payload contains any special characters. We can click on the **Generate Preview** button to preview how our final payload will look in the request:



Once that's done, we can click on **Add** to add the processor and click on **Ok** in the processors and payloads windows to close them.

## Options

Finally, we can set a few options for our fuzzers, similar to what we did with Burp Intruder. For example, we can set the **Concurrent threads per scan** to **20**, so our scan runs very quickly:



The number of threads we set may be limited by how much computer processing power the server allows us to establish.

We may also choose to run through the payloads **Depth first**, which would attempt position before moving to the next (e.g., try all passwords for a single user before br

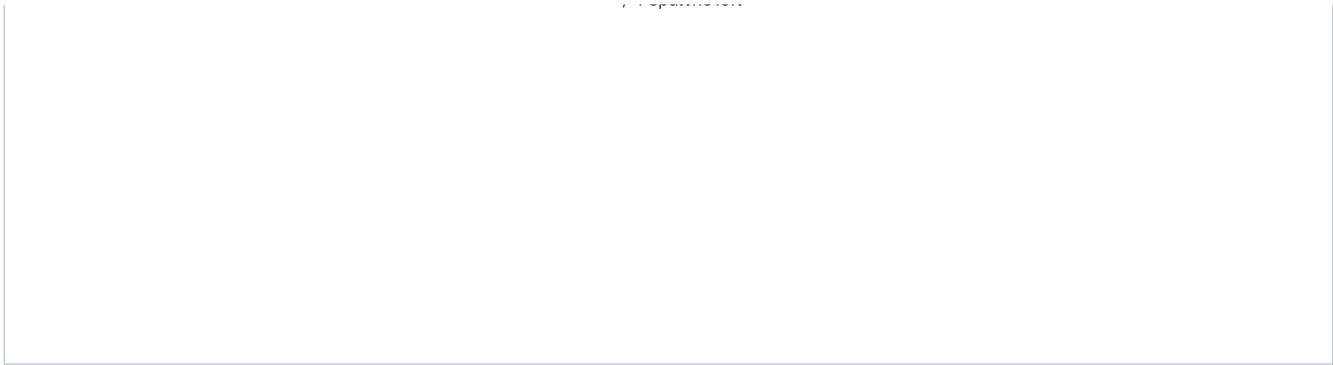
**Breadth first**, which would run every word from the wordlist on all payload positions before moving to the next word (e.g., attempt every password for all users before moving to the following password).

**Almost There!** BETA X

10 Streak pts left to meet The Weekly Goal.

20 / 30 Streak pts earned





Waiting to start...

## Questions

 Cheat Sheet

Answer the question(s) below to complete this Section and earn cubes!

Target: 83.136.252.32:37679 

Life Left: 47 minute(s)

**+2**  The directory we found above sets the cookie to the md5 hash of the username, as we can see the md5 cookie in the request for the (guest) user. Visit '/skills/' to get a request with a cookie, then try to use ZAP Fuzzer to fuzz the cookie for different md5 hashed usernames to get the flag. Use the "top-usernames-shortlist.txt" wordlist from Seclists.

HTB{fuzz1n6\_my\_f1r57\_c00k13}

+10 Streak pts

 Submit

 Hint

 Show Solution

 Previous

Next 

+10 Streak pts

 Mark Complete & Next

 Cheat Sheet

 Go to Questions

## Table of Contents

### Getting Started

Intro to Web Proxies 

Setting Up 

### Web Proxy

Proxy Setup 

 Intercepting Web Requests 

Intercepting Responses

Automatic Modification

 Repeating Requests

 Encoding/Decoding

 Proxying Tools

### Web Fuzzer

 Burn Intruder 

## Almost There!

BETA



10 Streak pts left to meet The Weekly Goal.

20 / 30 Streak pts earned

 Burp Intruder ZAP Fuzzer

## Web Scanner

Burp Scanner

 ZAP Scanner

Extensions

## Skills Assessment

 Skills Assessment - Using Web Proxies

## My Workstation

OFFLINE

 Start Instance / 1 spawns left**Almost There!** BETA

10 Streak pts left to meet The Weekly Goal.

**20** / 30 Streak pts earned